

Nhật ký thiết kế test plan — từng bước

- Sinh viên: Lê Nhựt Duy — MSSV: 23127178
- Phụ lục của: `ai-audit-report.md` · Quy trình: `.claude/skills/perf-test-plan/SKILL.md`

§2 đòi: "Drive an AI tool — step by step, not with a single generic prompt."

`ai-audit-report.md` ghi **prompt nguyên văn** của sinh viên — và những prompt đó ngắn. File này ghi **bảy bước bên trong** mà quy trình thực sự đi qua để ra được 4 test plan: mỗi bước hỏi gì, căn cứ nào, quyết ra sao, và **thay đổi cụ thể nào** trong file.

Ranh giới phải nói rõ để không hiểu nhầm: cột "Quyết định" dưới đây là quyết định của quy trình AI đi theo skill, **không phải** prompt do sinh viên gõ. Prompt do sinh viên gõ nằm nguyên văn ở `ai-audit-report.md`, không sửa. Hai thứ được tách ra để người chấm thấy đúng cái gì là của ai — trộn chúng lại thành "sinh viên đã hỏi 7 câu" sẽ là bịa.

Bước 1 — Nạp ngữ cảnh SUT trước khi chốt bất kỳ tham số nào

Hỏi gì: ba đặc điểm nào của SUT sẽ làm một test plan generic trở nên sai?

Căn cứ: đọc `backend/server.js`, 31 route.

Phát hiện	Vị trí	Ảnh hưởng tới thiết kế
Backend là một process Node, một luồng JS + SQLite một writer	toàn file	Tăng VU quá một mức chỉ làm dài hàng đợi. Trần thật là ~100% của một lỗi
Lockout kích hoạt sau 2 lần sai, không phải 3 — <code>login_attempts + 2</code> , ngưỡng 3	<code>server.js:54-58</code>	Mọi nhánh "login sai" phải dùng tài khoản tách riêng , và phải reset giữa các lượt
Không có rate limiting ở bất kỳ route nào	toàn file	Mọi 4xx đến từ credential/token/body — không được giải thích bằng "server tự bảo vệ"
Login có ghi DB: <code>UPDATE users SET login_attempts=0</code>	<code>server.js:47</code>	Endpoint auth không read-only → mỗi VU phải có tài khoản riêng, nếu không sẽ đo write-contention do chính mình tạo ra
<code>PUT /orders/:id/status</code> là state machine FR-10	<code>server.js:537-551</code>	Một order chỉ chuyển tiếp một lần/trạng thái → assert cứng 200 sẽ sai

Quyết định: không bắt đầu sinh `.jmx` cho tới khi 5 điều trên được viết ra. Bốn trong số đó về sau trực tiếp gây ra lỗi khi bị bỏ qua (bước 5 và 6).

Bước 2 — Chốt tham số từng scenario, mỗi con số một lý do

Hỏi gì: mỗi scenario phải trả lời được **câu hỏi nào**, và tham số nào phục vụ đúng câu hỏi đó?

Sce nari o	Câu hỏi phải trả lời	Tham số chốt	Vì sao con số đó
Loa d	p95 ở tải kỳ vọng?	20 VU · ramp 60s · think 1–3s/iteration · 360s	Think-time là thứ phân biệt "tải kỳ vọng" với stress. Không có nó thì mỗi VU thành vòng lặp bận và 20 VU sinh tải như hàng trăm người
Stre ss	Điểm gãy ở mức VU nào?	bậc 25 → 50 → 100 → 200 · mỗi bậc 30s ramp · 480s	Nhảy thẳng 200 VU thì mọi thứ đổ vỡ cùng lúc, chỉ biết "có gãy" chứ không biết gãy ở đâu
Spik e	Bao lâu thì hồi phục?	10 VU nền chạy xuyên lượt + 200 VU trong 5s ở giây 60 · 240s	Nhánh nền là bắt buộc. Không có nó thì sau cú sốc không còn gì để đo, chỉ thấy "lúc sốc thì chậm" — điều hiển nhiên
Soa k	Có trôi p95 / rò rỉ?	20 VU · 720s	§6 đòi 10–15 phút. Dưới 10 phút không gọi là soak

Quyết định về timer: dùng **Uniform Random Timer**, không dùng hằng số — hằng số làm mọi VU đồng bộ thành từng đợt và tạo đỉnh tải giả.

Bước 3 — Sinh `.jmx` bằng script, không viết tay

Hỏi gì: làm sao **đảm bảo** cả 3 plan chạy cùng một workflow, chứ không phải "cố gắng cho giống"?

Quyết định: viết `tools/gen-test-plans.py` — workflow định nghĩa **một lần** trong hằng `WORKFLOW` (dòng 29), 4 plan phát ra từ nó; chỉ `SCENARIOS` (dòng 107) khác nhau.

Vì sao không viết tay 4 file XML: 400–1100 dòng mỗi file. Sớm muộn cũng lệch nhau một assertion, và lúc đó so Load với Stress với Spike mất ý nghĩa vì không còn đo cùng một thứ. §6 đòi "*the same end-to-end workflow*" — script biến điều đó từ **lời hứa** thành **tính chất không thể vi phạm**.

Tên file: `{MSSV}_{Scenario}_{YYYYMMDD}.jmx` ngay từ đầu, không đổi tên sau — §11 kiểm tay đúng cái tên này, và đổi tên sau khi chạy là chỗ dễ lệch giữa `.jmx`, `.jtl` và báo cáo.

Bước 4 — Assertion: kiểm cả mã lỗi lẫn nội dung

Hỏi gì: assertion nào bắt được lỗi thật, và assertion nào tạo ra **kết quả tốt hơn thực tế**?

Bước	Assert	Vì sao không lỏng hơn / chặt hơn
1 login	200 và body chứa token	200 mà thiếu token thì 4 bước sau vô nghĩa nhưng vẫn "pass"
2, 3 GET	200	401/403 trả gần như tức thì → chỉ đo thời gian sẽ ra p95 đẹp giả
4 import	200 và có field message	Không assert theo <code>inserted</code> : con số đó phụ thuộc dữ liệu (dòng CSV thiếu <code>name</code> bị bỏ qua hợp lệ)
5 status	<code>200\ 400</code>	FR-10 chỉ cho chuyển tiếp một lần → 400 là phản hồi đúng

Bước	Assert	Vì sao không lỏng hơn / chặt hơn
6 lockout	40[13]	401 ở lần sai đầu, 403 sau khi đã khoá — cả hai đều đúng, tùy thời điểm

Nguyên tắc rút ra ở bước này: assertion yếu nguy hiểm hơn assertion sai, vì nó làm kết quả **tốt hơn** thực tế — request bị từ chối trả về nhanh, kéo p95 xuống.

Bước 5 — Human review trên `.jmx` do AI sinh

Checklist 8 mục của skill, chạy trước khi bấm chạy lượt nào:

- [x] Có think-time chưa? Loại timer nào? → **CÓ**, Uniform Random
- [x] Ramp-up tương xứng số thread? → **CÓ**
- [x] Token lấy từ JSON Extractor, không hard-code? → **CÓ**, `$.token` → `${TOKEN}`
- [x] CSV `recycle=true` ? shareMode đúng? → **CÓ**, `shareMode.all`
- [x] Assertion kiểm nội dung hay chỉ status? → **cả hai** ở bước 1 và 4
- [x] Có nhánh login sai? Dùng **tài khoản riêng** chưa? → có nhánh; **tài khoản riêng thì BỎ SÓT** → lỗi #5
- [x] Listener trùng loại giữa 3 plan? → không trùng
- [x] View Results Tree bật ở lượt dài? → không, chỉ ở Spike

Bước này bắt được 1 lỗi. Nó bỏ sót 3 lỗi mà chỉ chạy thật mới lộ — đó là lý do bước 6 không được bỏ.

Bước 6 — Smoke test 20–40 giây trước khi chạy lượt 6 phút

Hỏi gì: plan có **đo đúng** không — chứ không phải plan có **chạy được** không?

Lượt smoke	Quan sát	Chẩn đoán	Sửa
20s, 2 VU	41,38% error	Nhánh lockout: assertion regex pass, nhưng JMeter đã gắn cờ Fail cho 4xx từ trước và assertion chỉ thêm được lỗi, không xoá được cờ	JSR223PostProcessor gọi <code>prev.setSuccessful(true)</code> khi mã khớp <code>40[13]</code>
20s, 2 VU	Bước 5 trả 400	<code>orders.csv</code> xen kẽ <code>shipping</code> , mà từ <code>pending</code> thì <code>shipping</code> không hợp lệ	<code>orders.csv</code> chỉ dùng <code>confirmed</code> ; thêm <code>reset-orders.mjs</code>
40s, 10 VU	0% error	—	đủ điều kiện chạy lượt thật

Bước 7 — Đọc kết quả trước khi tin nó

Ba câu của skill, hỏi trên chính lượt chạy thật:

1. Error % có bao nhiêu phần là chức năng, bao nhiêu là hiệu năng?

Lượt Load thật đầu tiên: **2,9% error**, và toàn bộ là 403. Truy ra: `users.csv` chứa 2 dòng mật khẩu sai, luồng chính đọc chính file đó với `recycle=true` → **tự khoá tài khoản của mình** → 4 bước sau của iteration đó cũng 403. Không sample nào trong số đó nói gì về SUT. → **Huỷ lượt chạy, xoá sạch bằng chứng**, tách `users_lockout.csv`.

2. Peak thread thực tế có bằng cấu hình không?

Có — nhưng throughput chỉ ~10 sample/s thay vì ~50. Truy ra: timer đặt ở scope thread group nên JMeter chen nó trước **từng** sampler — 5 lần một iteration, tức 5–15 giây/iteration. → **Huỷ lượt thứ hai**, chia lại think-time còn 200–600ms/bước để **tổng** đúng 1–3s.

3. p95 của từng endpoint, hay chỉ p95 tổng?

Từng endpoint. Và chính điều này lộ ra rằng `POST /api/login` (107ms) đắt hơn p95 tổng (70ms) **1,53 lần** — một hồi quy ở login sẽ không làm p95 tổng nhích lên.

Lượt thứ ba còn lộ thêm: **18,25% error**, toàn bộ là 400 hợp lệ của FR-10 ở bước 5 — **đúng cơ chế đã sửa cho nhánh lockout ở bước 6 mà quên áp cho bước 5**.

Bảy bước này đã trả lại cái gì

	Số
Lỗi bắt được trước khi chạy (bước 5)	1
Lỗi chỉ smoke test mới lộ (bước 6)	2
Lỗi chỉ lượt chạy dài mới lộ (bước 7)	3
Lỗi trong tooling, lộ khi đọc dữ liệu	4
Tổng	10
Trong đó không làm test plan báo lỗi	7
Lượt chạy phải huỷ và xoá bằng chứng	2

Con số đáng chú ý nhất: **7/10 lỗi không làm plan báo lỗi**. Plan vẫn chạy, vẫn sinh `.jtl`, vẫn ra dashboard trông chuyên nghiệp — chỉ con số là sai. Nếu quy trình chỉ có "sinh plan rồi chạy" mà không có bước 5–6–7, cả 7 lỗi đó đều đi thẳng vào báo cáo.

Danh sách 10 lỗi kèm phân loại lý do: [ai-audit-report.md](#) và [report/main-report.md §2.4](#).